Theory

1. Consider the array [8, 2, 5, 1, 9, 7, 8, 3]. Write down the modified array when it will be converted to a Minheap. Again show the final array once extractMin() has been called on the previous modified array.

Ans:

Min Heap: [1, 2, 5, 3, 9, 7, 8, 8]

After extractMin(): [2, 3, 5, 8, 9, 7, 8]

2. True or False:

A. A heap is always a complete binary tree (True)

B. Heap supports fast searching of arbitrary elements (False)

C. Heap can be represented using an array (True)

3. If a Max heap is constructed with 45 nodes, how many leaf nodes will it contain?

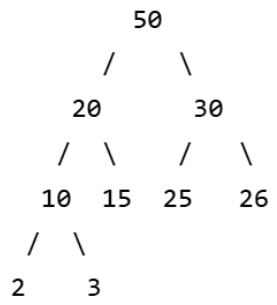
Ans:

Number of non-leaf nodes = $\lfloor n / 2 \rfloor$

Number of leaf nodes = $n - \lfloor n / 2 \rfloor$

Leaf nodes=45- $\lfloor 45/2 \rfloor$= 45- $\lfloor 22.5 \rfloor$= 23

4.



2. From the given Max Heap tree, first remove 20 and 3. Then add 12 in that tree. Draw the final executed Max Heap tree. [2 points]

Answer:

5. What's the time complexity of the Swim (heapify-up) operation in a heap?

a) O(n)

b) O(n log n)

c) O(n^2)

d) O(log n)

Answer:
✅ d) O(log n)

6. Which operation requires Heapify?
Ans: Deleting

7. Insert 10, 5, 20, 15, 17 one after another in a max-heap and draw the max-heap after each insertion.

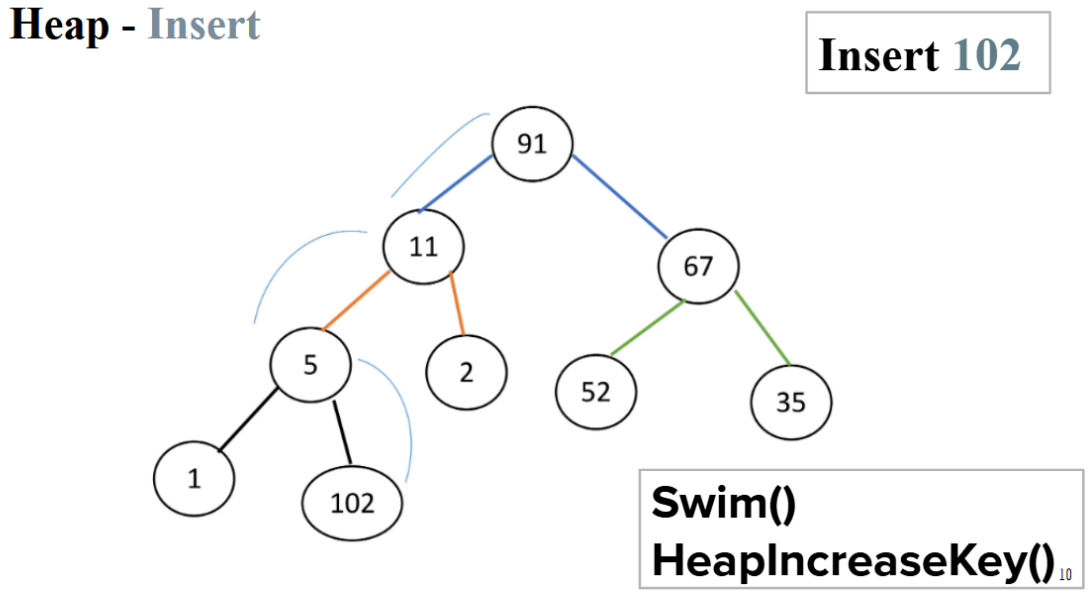
10	10 / 5	Before swim: 10 / \ 5 20 After swim: 20 / \ 5 10	Before swim: 20 / \ 5 10 / 15 After swim: 20 / \ 15 10 / 5	Before Swim: 20 / \ 15 10 / \ 5 17 After swim: 20 / \ 17 10 / \ 5 15

8. Which traversal of a heap always gives elements in sorted order?
a) Inorder
c) Postorder
b) Preorder
d) None of these

Ans: (d)
Because inorder, preorder, and postorder traversals of a heap do not produce sorted order; only repeated deletion (heap sort) gives sorted elements.
Inorder gives sorted in- BST, not heap

Basics

Heap- Abstract data type which is a Binary tree



(For Max Heap, Min Heap code will be similar)

```
public static void insert(int key) {
    size = size + 1;
    H[size] = key;
    swim(size);
}
```

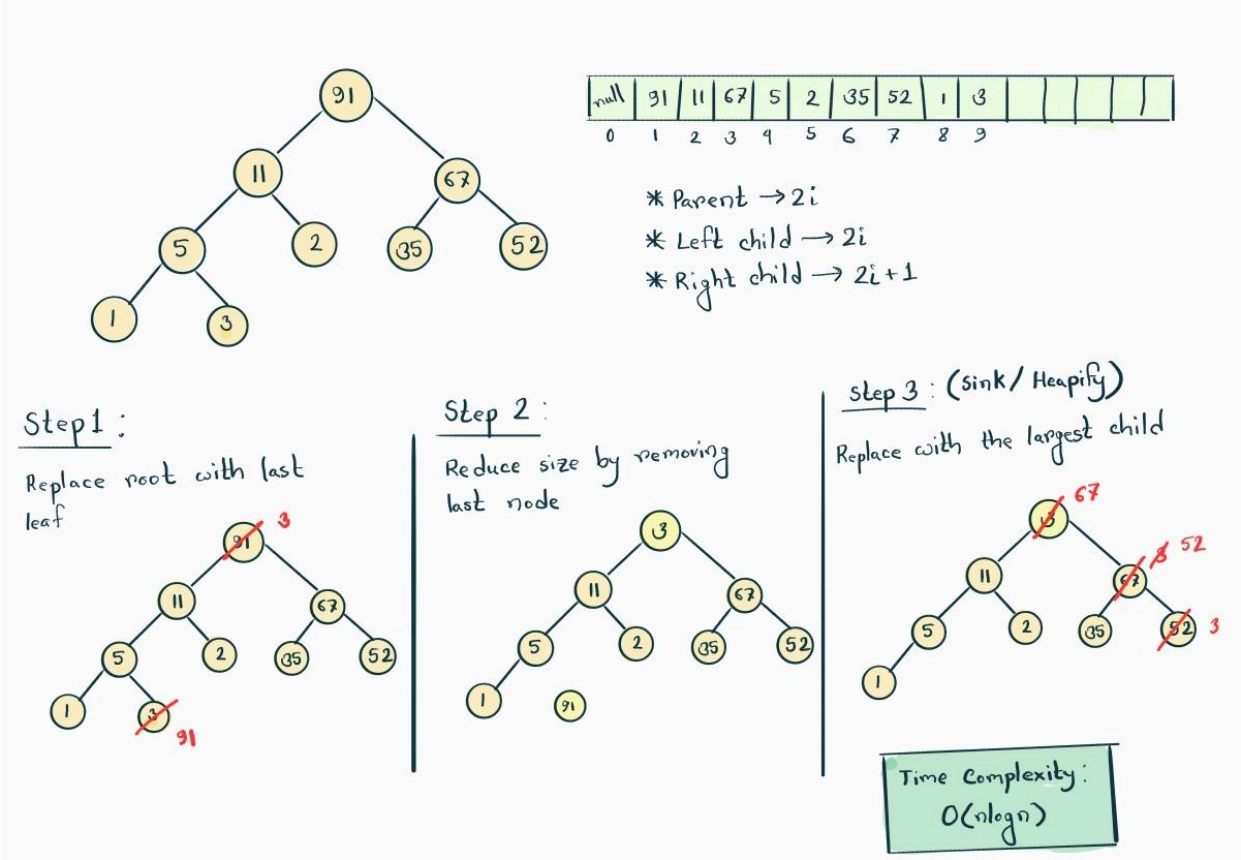
```
private static void swim(int index) {
    if (index <= 1) return;

    int parent = index / 2;
    if (H[parent] > H[index]) return;

    int temp = H[parent];
    H[parent] = H[index];
    H[index] = temp;
```

```
    swim(parent);
}
```

Delete (Extract Max)



(For Max Heap, Min Heap code will be similar)

```
public Integer ExtractMax(){
    if(size==0) return null;

    Integer extractMax= maxHeap[1];
    maxHeap[1]=maxHeap[size];
    size--;
    maxHeapify(1);
```

```
        return extractMax;
    }



//Max heapify/ Sink
    private void maxHeapify(int indx){
        int parent=indx;


        while(2*parent<= size){
        int leftChild= 2*parent;
        int rightChild= 2*parent +1;
        int largerChild= leftChild;

        if(rightChild<= size && maxHeap[rightChild]> maxHeap[leftChild]){
            largerChild= rightChild;
        }

        if(maxHeap[parent]>= maxHeap[largerChild]){
            return;
        }

        swap(parent, largerChild);
        parent= largerChild;
        }

    }



//Swap
    private void swap(int i, int j){
        int temp= maxHeap[i];
        maxHeap[i]= maxHeap[j];
        maxHeap[j]= temp;
    }
```

Sort (For Max Heap, Min Heap code will be similar)

```
private int[] sort(){
    int[] backup= maxHeap.clone();        //Clone na korle address store hobe just
    int ogSize=size;


    int[] sorted= new int[ogSize];
    for(int i = size-1; i >= 0; i--){          //Max heap sort = Descending order
        sorted[i]= deleteMax();
    }


    maxHeap= backup;
    size=ogSize;
    return sorted;
}
```

Lab Task[No need]

Task 1: MinHeap Class

```
public class MinHeap {
    private int[] minHeap;
    private int size;

    public MinHeap(int length){
        minHeap= new int[length +1];
        size=0;
    }


    //Insert
    public void insert(int elem){
        if(size<minHeap.length-1){
            size+=1;
            minHeap[size]= elem;
            swim(size);
        }

        else{
            System.out.println("Heap is full");
        }
    }


    private void swim(int size){
        int child=size;
        int parent= child/2;

        while(parent>0 && minHeap[parent]> minHeap[child]){       //0 index restricted
            swap(parent,child);
            child=parent;
            parent=child/2;
        }
    }

    private void swap(int i, int j){
        int temp= minHeap[i];
        minHeap[i]= minHeap[j];
        minHeap[j]= temp;
    }
```

```
//Delete
public Integer delete(){
    if(size==0) return null;

    Integer extractMin= minHeap[1];
    minHeap[1]= minHeap[size];
    size--;

    minHeapify(1);

    return extractMin;
}



private void minHeapify(int indx){
    int parent=indx;

    while(2*parent<=size){
    int leftChild= 2*parent;
    int rightChild = 2*parent + 1;
    int smallerChild = leftChild;

    if(rightChild<=size && minHeap[rightChild]< minHeap[leftChild]){
        smallerChild= rightChild;
    }

    if(minHeap[parent] <= minHeap[smallerChild]){
        return;
    }

    swap(parent, smallerChild);
    parent=smallerChild;

    }



}
```

```
    //Sort
    private int[] sort(){
        int[] backup= minHeap.clone();
        int ogSize=size;

        int[] sorted= new int[ogSize];
        for(int i = size-1; i >= 0; i--){
            sorted[i]= delete();
        }

        minHeap= backup;
        size=ogSize;
        return sorted;
    }



    //Printing
    public void printArr(){
        for(int i=0; i< minHeap.length;i++){
            System.out.print(minHeap[i]+ " ");
            if(i==size){
                System.out.print(" || ");
            }
        }
    }
}
```

Task 2: MaxHeap Class

```
public class MaxHeap{
    private int[] maxHeap;
    private int size;

    public MaxHeap(int length){
```

```
    maxHeap= new int[length +1];
    size=0;
}


//Insert
public void insert(int elem){
    if(size<maxHeap.length-1){
        size+=1;
        maxHeap[size]= elem;
        swim(size);
    }

    else{
        System.out.println("Heap is full");
    }
}


private void swim(int size){
    int child=size;
    int parent= child/2;

    while(parent>0 && maxHeap[parent]< maxHeap[child]){       //0 index restricted
        swap(parent,child);
        child=parent;
        parent=child/2;
    }
}

private void swap(int i, int j){
    int temp= maxHeap[i];
    maxHeap[i]= maxHeap[j];
    maxHeap[j]= temp;
}




//Delte
private Integer delete(){
    Integer extractMax= maxHeap[1];
    maxHeap[1]=maxHeap[size];
```

```
    size--;

    sink(1);

    return extractMax;
}

private void sink(int indx){
    if(indx>=size) return;

    int parent=indx;
    int leftChild= 2*indx;
    int rightChild= 2*indx +1;

    if(parent<size && maxHeap[parent]<maxHeap[leftChild]){
        swap(parent,leftChild);
    }

    if(parent<size && maxHeap[parent]<maxHeap[rightChild]){
        swap(parent,rightChild);
    }


}



//Sort
private int[] sort(){
    int[] backup= maxHeap.clone();
    int ogSize=size;

    int[] sorted= new int[ogSize];
    for(int i = size-1; i >= 0; i--){
        sorted[i]= deleteMax();
    }

    maxHeap= backup;
    size=ogSize;
    return sorted;
}
```

```
    //Printing
    public void printArr(){
        for(int i=0; i< maxHeap.length;i++){
            System.out.print(maxHeap[i]+ " ");
            if(i==size){
                System.out.print(" || ");
            }
        }
    }
}
```

```
public class CusMaxHeap {
    String name;
    int priority;

    CusMaxHeap(String n, int p){
        name=n;
        priority=p;
    }
}



class MaxHeap{
    private CusMaxHeap[] maxHeap;
    private int size;

    public MaxHeap(int length){
        maxHeap= new CusMaxHeap[length+1];
        size=0;
    }



    //Insert
    public void insert(String name, int priority){
        if(size<maxHeap.length-1){
```

```
            size+=1;
            maxHeap[size]= new CusMaxHeap(name, priority);
            swim(size);
        }

        else{
            System.out.println("Heap is full");
        }
    }


    private void swim(int size){
        int child=size;
        int parent= child/2;

        while(parent>0 && maxHeap[parent].priority< maxHeap[child].priority){       //0 index
restricted
            swap(parent,child);
            child=parent;
            parent=child/2;
        }
    }

    private void swap(int i, int j){
        CusMaxHeap temp= maxHeap[i];
        maxHeap[i]= maxHeap[j];
        maxHeap[j]= temp;
    }




    //Delte
    public String deleteMax(){
        if(size==0) return null;

        String extractMax= maxHeap[1].name;
        maxHeap[1]=maxHeap[size];
        size--;
        sink(1);

        return extractMax;
    }
```

```
    private void sink(int indx){
        int parent=indx;

        while(2*parent<= size){
        int leftChild= 2*parent;
        int rightChild= 2*parent +1;
        int largerChild= leftChild;

        if(rightChild<= size && maxHeap[rightChild].priority> maxHeap[leftChild].priority){
            largerChild= rightChild;
        }

        if(maxHeap[parent].priority>= maxHeap[largerChild].priority){
            return;
        }

        swap(parent, largerChild);
        parent= largerChild;
        }

    }
}
```

Lab Assignment

You are given an array of n tasks, where the value of index i represents the processing time for i-th task. You are also given m machines, each capable of processing one task at a time. Your goal is to distribute the tasks among the machines such that the overall completion time for each machine is minimized. Represent the output as an array where each index corresponds to a machine, and the value at each index is the total processing time of tasks assigned to that machine.

Input Format:

- **tasks:** An array of integers where each integer represents the processing time of a task.
- **m:** An integer representing the number of machines.

Output Format:

- An array of integers of size m, where the value at index i represents the total processing time of tasks assigned to the i-th machine.

Sample Given Input:
tasks = [2, 4, 7, 1, 6]
m = 4

Sample Output:
[2, 4, 7, 7]

```
private static Integer[] machineTask(Integer[]tasks,int m){
    MinHeap mH= new MinHeap(tasks.length);

    //Inserting and then taking out the minimum
    for(int i=0; i<tasks.length; i++){
        mH.insert(tasks[i]);
    }
    Integer min= mH.delete();


     //Creating another Heap array with all zeros
    MinHeap newHeap=new MinHeap(tasks.length);
    for(int i=0; i<m; i++){
        newHeap.insert(0);
    }

    //Inputing all the ones till length
    int change= tasks.length-m;
    for(int i=0; i<change; i++){
        newHeap.delete();
        newHeap.insert(tasks[i]);
    }
```

```
    //Adding the extra one
    for(int i=0; i<tasks.length-change; i++){
        Integer store= newHeap.delete();
        Integer newAdd= store + tasks[change+i];
        newHeap.insert(newAdd);
    }

    //Converting to integer
    Integer[] arr2=new Integer[m];
    for(int i=0; i<m; i++){
        arr2[i]= newHeap.delete();
    }
    return arr2;
}
```

Largest Elements in Descending Order

You are given an array of integers and a number k. Your task is to find the top k largest elements from the list. Use a max-heap to solve this problem efficiently.

Input Format:

- nums: An array of integers.
- k: An integer representing the number of largest elements to find.

Output Format:

- An array of k integers representing the largest elements in descending order.

Sample Input:
nums = [4, 10, 2, 8, 6, 7]
k = 3
Output:
[10, 8, 7]

```
public static Integer[] largestDesArray(Integer[] nums, int k){

    //Filling up max heap
    MaxHeap H= new MaxHeap(nums.length);
     for(int i=0; i<nums.length; i++){
       H.insert(nums[i]);
    }


    //Filling up new array
    Integer[] arr2= new Integer[k +1];
    for(int i=0; i<arr2.length; i++){
        arr2[i]= H.deleteMax();
    }

    return arr2;
}
```

You are given two arrays: one containing task names and another containing their corresponding priorities. Your task is to process all tasks in order of their priority using a Max-heap. Tasks with higher priority values should be processed first.

Input Format:

- *task_names:* An array of strings where each element represents a task name.
- *priorities:* An array of integers where each element represents the priority of the corresponding task.

Output Format:

- An array of strings representing the task names in the order they should be processed (highest priority first).

Steps to Solve:

1. Build a customized max-heap where each element contains both the task name and its priority.
2. Iterate through both arrays simultaneously and insert each (task_name, priority) pair into the max-heap (heap should compare based on priority).
3. Extract the maximum repeatedly until the heap is empty, storing each task name in the result array.
4. Return the result array containing all task names in priority order.

Sample Input
task_names = ["Email", "Meeting", "Code Review", "Lunch", "Debug"] priorities = [2, 5, 3, 1, 4]
Sample Returned Priorities Array
["Meeting", "Debug", "Code Review", "Email", "Lunch"]
Explanation
To solve this problem, we use a max-heap like a priority queue because it allows efficient retrieval of the highest priority task. The process involves: 1. Combine task names with their priorities and insert all tasks into a max-heap based on their priority values. 2. Create a result array to store task names in processing order. 3. Extract the maximum (highest priority task) repeatedly until the heap is empty. 4. Return the result array containing task names in descending order of priority.

```
//Here a  special HeapMax is taken that will take both int and String
//swim will happen based on priority(int)
//swap will happen based on priority(int). CusHEapMax is being swapped
// Delete will return String

    public static String[] priorityArr(String[] taskName, int[] priorities){
        MaxHeap H= new MaxHeap(taskName.length);

        for(int i=0; i<taskName.length; i++){
            H.insert(taskName[i], priorities[i]);
        }


        String[] arr2= new String[taskName.length];
        for(int i=0; i<taskName.length; i++){
            arr2[i]= H.deleteMax();
```

```
    }

    return arr2;
}
```

Lab Exam

Min Heap to Max Heap conversion

A teacher uses an automated grading system that stores students' exam scores in a min-heap (*with the lowest score at the top*) for efficient processing. Before publishing the ranking from highest to lowest, the teacher decides to **add one grace mark** to every student's score (*capping the maximum at 100*) and then convert the updated scores from **a min-heap to a max-heap** so the highest score is at the top.

Now, write a function, **rescue(n, scores)**, that takes *n* as the number of students' scores in the list and *scores* as a list of *n* integers representing the students' scores in min-heap order.

Sample Input	Sample Output	Hint
n = 5 **scores = [56, 59, 69, 89, 100]**	**[100, 90, 70, 60, 57]**	Implement your own *heapify* function and conversion logic to build a *max-heap*. Do not create a new array for the *max-heap*. Perform the heap conversion **in-place**. Your function should return the *max-heap* array after conversion, not print it.

[You are not allowed to use any built-in functions and heap libraries]

```
public static int[] rescue(int n, int[] scores) {

    // Step 1: Add grace mark (cap at 100)
    for (int i = 0; i < n; i++) {
        if (scores[i] < 100) {
            scores[i] = scores[i] + 1;
        }
    }



    // Step 2: Convert to Max Heap
    for (int i =0; i <(n / 2) - 1; i++) {
        maxHeapify(scores, n, i);
    }

    return scores;
}



private static void maxHeapify(int[] arr, int n, int i) {
    int largest = i;
    int left = 2 * i;
```

```
        int right = 2 * i + 1;

        if (left < n && arr[left] > arr[largest]) {
            largest = left;
        }

        if (right < n && arr[right] > arr[largest]) {
            largest = right;
        }

        if (largest != i) {
            int temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;

            maxHeapify(arr, n, largest);     //Recursive call only when largest value isn't the
parent
        }
    }
}
```

A sports academy is compiling the performance scores of its athletes. Due to a sensor malfunction, some scores have been mistakenly recorded as negative numbers. Since a score cannot be negative, these values should be treated as their absolute values. Your task is to write a function **sortScores()** that takes an array of integers representing the athlete's scores. Then sort the scores in descending order and return the modified sorted array.

- The solution must not exceed **O(n.log(n))** time complexity.
- You can use **MinHeap** data structure.
- You can not take additional arrays or other data structures except Heap.

The **MinHeap** class has already been defined for you. You can create an instance of this class and use the public functions **insert**() and **extract**(). When creating the heap, you will need to specify its capacity. *No need to write the MinHeap class on your own.*

Sample Input	Sample Output	Explanation
[50, -20, 75, -10, 90, 30]	[90, 75, 50, 30, 20, 10]	All the numbers are treated as absolute values and then sorted in-place using a MinHeap.
[40, -40, 25, -25, 40, 10]	[40, 40, 40, 25, 25, 10]	

```
//java function signature
public static int[] sortScores(int[] scores){
```

```
#python function signature
def sortScores(scores):
```

```java
public static int[] sortScores(int[] scores) {

    MinHeap heap = new MinHeap(scores.length);

    // Insert absolute values into heap
    for (int i = 0; i < scores.length; i++) {
        int value = scores[i];
        if (value < 0) {
            value = -value;
        }
        heap.insert(value);
    }

    // Extract from heap and fill array from end (descending order)
    for (int i = scores.length - 1; i >= 0; i--) {
        scores[i] = heap.extract();
    }

    return scores;
}
```

In an F1 qualifying session, each driver completes two laps.
You are given three arrays:

drivers → the names of drivers
lap1[] → time taken in the first lap
lap2[] → time taken in the second lap

A driver's best lap time (minimum of lap1 and lap2) is considered their priority.
Smaller lap time = higher priority

You must organize the drivers using the appropriate heap so that they are ranked from fastest to slowest, based on their best lap times.
Constraints:

- You cannot use any built-in functions except len().
- Direct sorting or searching methods on the array are not allowed.
- Assume a heap class is created and heap-related functions **extractMin(),extractMax(), sink(), swim()** are implemented and can be used while other necessary functions must be implemented. Assume the heap class has two arrays to store the best lap and names.

Sample Input:	**Sample Output:**
drivers = ["Verstappen", "Hamilton", "Leclerc", "Norris", "Russell"] lap1 = [75, 78, 77, 79, 80] lap2 = [76, 74, 78, 80, 79]	Result = ["Hamilton", "Verstappen", "Leclerc", "Norris", "Russell"]
Explanation:	
Each driver has two lap times, and their **best lap** (the smaller time) determines their ranking. Faster drivers have lower lap times, so we rank them from **fastest to slowest** based on their best lap. For example, Hamilton's best lap is 74 seconds and Verstappen's is 75, so Hamilton is ranked higher.	

(Priority Queue)

```
public static String[] rankDrivers(String[] drivers, int[] lap1, int[] lap2) {

    int n = drivers.length;
    MinHeap heap = new MinHeap(n);

    // Step 1: insert drivers with best lap
    for (int i = 0; i < n; i++) {
        int bestLap;
```

```
        if (lap1[i] < lap2[i]) {
            bestLap = lap1[i];
        } else {
            bestLap = lap2[i];
        }

        heap.insert(bestLap, drivers[i]);
    }

    // Step 2: extract in order (fastest to slowest)
    String[] result = new String[n];
    for (int i = 0; i < n; i++) {
        result[i] = heap.extractMin();
    }

    return result;
}
```

In applications like task scheduling, it's vital to manage tasks based on their urgency, represented by priority levels where a higher number signifies higher urgency. You are given two arrays: one for task IDs and the other for corresponding priorities. These tasks must be organized using a data structure designed for efficient access to the most urgent tasks. However, only those tasks within a specified priority range, marked by lowPriority and highPriority, should be considered. Tasks falling outside this range should be excluded.
[Note: The most urgent tasks have higher priority numbers, so the data structure should focus on retrieving the maximum values within the range to correctly identify and process the highest-priority tasks]

Note:

1. You cannot use any built-in functions except len().
2. Direct sorting or searching methods on the array are not allowed.
1. Assume heap-related functions **extractMax(), sink()** are implemented and other necessary functions must be implemented.

Sample Input:	**Sample Output:**
tasks = [101, 102, 103, 104, 105, 106, 107, 108, 109] priorities = [9, 3, 7, 1, 5, 8, 2, 6, 4] lowPriority = 3 highPriority = 7	result = [103, 108, 105, 109, 102]
Explanation:	
Given lowPriority = 3 and highPriority = 6, tasks with the third, fourth, fifth, and sixth highest priorities based on the priority scale are selected. This corresponds to the tasks with priorities 7, 6, 5, and 4 after selecting them by their priority, which are tasks 103, 108, 105, and 109 respectively.	

```
public static int[] getTasksInPriorityRange(int[] tasks, int[] priorities, int lowPriority, int highPriority) {

    int n = tasks.length;
    MaxHeap heap = new MaxHeap(n);

    // Step 1: build heap
    for (int i = 0; i < n; i++) {
```

```
        heap.insert(priorities[i], tasks[i]);
    }




    int resultSize = highPriority - lowPriority + 1;
    int[] result = new int[resultSize];

    int extractCount = 0;
    int resultIndex = 0;

    // Step 2: extract by rank
    while (resultIndex < resultSize) {
        int taskId = heap.extractMax();
        extractCount++;

        if (extractCount >= lowPriority && extractCount <= highPriority) {
            result[resultIndex] = taskId;
            resultIndex++;
        }
    }

    return result;
}
```

You are given two arrays A and B, both consisting of n integers. Starting from index 0, for each element in B, you must subtract it from the current largest integer in A. After processing all elements in B, return array A sorted in non-increasing order.

You are only allowed to use the insert() and extract() method from MinHeap/MaxHeap. Do not use any other functions (for example, sort()).

```
def sub_arrays(n, A, B):
     //to do
```

Input	Output	Explanation
n = 3 A = [1, 3, 5] B = [2, 4, 6]	A = [1, -1, -3]	Initially, A=[1, 3, 5], B = [2, 4, 6] 1. Sub B[0]=2 from the largest integer A[2]=5. A becomes [1, 3, 3]. 2. Sub B[1]=4 from the largest integer A[2]=3. A becomes [1, 3, -1]. 3. Sub B[2]=6 from the largest integer A[1]=3. A becomes [1, -3, -1]. Finally, make A non-increasing to get [1, -1, -3].

```
public class SubArrays {

    public static int[] sub_arrays(int n, int[] A, int[] B) {
        MaxHeap heap = new MaxHeap(n);

        for (int i = 0; i < n; i++) {
            heap.insert(A[i]);
        }

        for (int i = 0; i < n; i++) {
            int max = heap.extract();
            max = max - B[i];
            heap.insert(max);
        }

        int[] result = new int[n];
        for (int i = 0; i < n; i++) {
            result[i] = heap.extract();
        }

        return result;
```

```
    }
}
```

Rfts slide

As a Software Engineer at Microsoft, you need to schedule tasks on the CPU based on priority. Given n tasks with unique priorities in an array and a value k, perform the first k tasks with the highest priority.

Implement the function **`cpu_scheduler(tasks, k)`** to print the task numbers of the top **k** tasks in descending order of priority. Use the provided MaxHeap or MinHeap classes with **`insert(item)`** and **`extract()`** (extract will return the extracted value) methods. **You cannot iterate through the array manually to find out the highest priority task. Check sample input-output to find out the printing format.**

You can create an Empty Heap using the following code	
Python Notation	Java Notation
`max_heap = MaxHeap()` `min_heap = MinHeap()`	`MaxHeap maxHeap = new MaxHeap()` `MinHeap minHeap = new MinHeap()`

Python Notation	Java Notation

`def cpu_scheduler(tasks, k):`
`  #To do`
	`static void cpu_scheduler(int[ ] tasks, int k){`
	`  //To do`
	`}`

Input	Output	Explanation

`tasks = [45, 70, 85, 60, 90, 75]`
`k = 3`
	`Task 1 - Priority 90`
	`Task 2 - Priority 85`
	`Task 3 - Priority 75`
		Here are the 6 tasks given in the array. The value of k is 3 so the function will print the first 3 tasks with higher priority according to the shown format.

```
static void cpu_scheduler(int[] tasks, int k) {

    MaxHeap heap = new MaxHeap(tasks.length);          //This class has both array and size

    for (int i = 0; i < tasks.length; i++) {      //Not i<size
        heap.insert(tasks[i]);
    }

    for (int i = 1; i <= k; i++) {
        int p = heap.extractMax();
        System.out.println("Task " + i + " - Priority " + p);
    }
}
```

Complete the function below that will take an array of tasks and print out the completion step of tasks based on their priorities. The value of the index of an array will represent the priority of that task. The lower the value, the higher the priority. One task will be done in one step. **Assume that all the values of the array will be unique, and you have a magical function `getIndex(arr, val)` which will return the index of that value in the array in constant time.**

The time complexity of your solution must be less than $O(n^2)$. You may use the provided MaxHeap or MinHeap classes with the **`insert(item)`** and **`extract()`** (extract will return the extracted value) methods. **Check the sample input-output to find out the printing format.**

You can create an Empty Heap using the following code	
Python Notation	Java Notation
`max_heap = MaxHeap()` `min_heap = MinHeap()`	`MaxHeap maxHeap = new MaxHeap()` `MinHeap minHeap = new MinHeap()`

Input	Output	Explanation

`tasks = [60, 85, 70, 45]`	`Step 1 - Task 4`
	`Step 2 - Task 1`
	`Step 3 - Task 3`
	`Step 4 - Task 2`
		Here, index 3 has the minimum value 45, so it will be done at step 1. `getIndex(tasks, 45)` will return 3, hence the task number is (3 + 1) = 4. The rest of the output is generated following the same pattern.

```
static void taskCompletion(int[] tasks) {

    MinHeap heap = new MinHeap(tasks.length);

    for (int i = 0; i < tasks.length; i++) {
        heap.insert(tasks[i]);
    }

    int step = 1;

    while (step <= tasks.length) {
        int val = heap.extractMin();
        int index = getIndex(tasks, val);
        System.out.println("Step " + step + " - Task " + (index + 1));
        step++;
    }
}
```

```
//get Index
public static int getIndex(int[] tasks, int val) {
    int i = 0;
    while (i < tasks.length) {
        if (tasks[i] == val) {
            return i;
        }
        i++;
    }
    return -1;
}
```

You are given an array A of size N. You can perform an operation in which you will remove the largest and the smallest elements from the array and add their difference back into the array. You are given an integer k that will define the number of times you have to perform the operation. Write a method heapSum() that takes an array, an integer k as parameters, performs k operations on the array, and returns the sum of the array elements after the operations.

You are given MaxHeap and MinHeap classes. The methods in the MaxHeap class are - MaxInsert(), MaxDelete() and the methods in the MinHeap class are - MinInsert(), MinDelete(). The constructor of both classes does not take any parameters. The method works according to their names and no need to implement any of the methods.

YOU MUST USE THE GIVEN HEAP DATA STRUCTURES TO SOLVE THIS PROBLEM.

Hint: No need to modify the given array. Remember that the underlying data structure of a heap is an array.

Python Notation:
```
def heapSum(A, k):
    # To Do
```

Sample Input	Sample Output	Explanation
heapSum(A, k) A=[3, 2, 1, 5, 4,] k = 2	9	After 1st operation, the array will become, A = [3,2,4,4]. After the 2nd operation, the array will become, A = [3,2,4]. sum of elements = 9. Here the shown value of A is just to explain the output and does not mean the state of the given array after each operation

```
static int heapSum(int[] A, int k) {

    MaxHeap maxHeap = new MaxHeap();
```

```
    MinHeap minHeap = new MinHeap();


    for (int i = 0; i < A.length; i++) {
        maxHeap.MaxInsert(A[i]);
        minHeap.MinInsert(A[i]);
    }


    for (int i = 0; i < k; i++) {
        int max = maxHeap.MaxDelete();
        int min = minHeap.MinDelete();

        int diff = max - min;

        maxHeap.MaxInsert(diff);
        minHeap.MinInsert(diff);
    }


    int sum = 0;

    while (!minHeap.isEmpty()) {
        sum += minHeap.MinDelete();
    }

    return sum;
}
```

Practise Problem

You are given an array of tasks, where each element represents the **urgency level** of that task and a deadline (number of days available). Your job is to complete the possible tasks within that deadline. However, there are a few rules:

- Each task takes **1 day** to complete.
- At each step, complete the task with the **highest urgency**.
- Ignore tasks left after the deadline.

The time complexity of your solution must be **less than $O(n^2)$. Assume that the classes MaxHeap and MinHeap are provided** with all necessary methods, including insert(value) and extract(), where extract() returns the removed element.

Caution: Using a Nested loop will result in $O(n^2)$ time complexity

You can create an Empty Heap using the following code	
Python Notation	**Java Notation**
`max_heap = MaxHeap()` `min_heap = MinHeap()`	`MaxHeap maxHeap = new MaxHeap()` `MinHeap minHeap = new MinHeap()`

Python Notation	Java Notation

`def urgent_task(tasks, deadline):`
`  #To do`
	`static void urgent_task(int[ ] tasks,int deadline){`
	`  //To do`
	`}`

Input	Output	Explanation

`tasks = [30, 95, 80, 25, 50]`
`deadline = 3`
	`Day 1 - Urgency 95`
	`Day 2 - Urgency 80`
	`Day 3 - Urgency 50`
	
	`Tasks completed: 3`
	`Tasks ignored: 2`
		Here, the deadline is 3 days and tasks 2, 3 and 5 have the highest urgency. So, they will be done in their respective urgency order. The rest of the tasks will be ignored.

```
static void urgent_task(int[] tasks, int deadline) {

    MaxHeap maxHeap = new MaxHeap();
    for (int i = 0; i < tasks.length; i++) {
        maxHeap.insert(tasks[i]);
    }


    int day = 1;
    while (day <= deadline && !maxHeap.isEmpty()) {
        int urgency = maxHeap.extract();
        System.out.println("Day " + day + " - Urgency " + urgency);
        day++;
    }

    System.out.println("Tasks completed: " + (day - 1));
    System.out.println("Tasks ignored: " + (tasks.length - (day - 1)));
}
```

You are given several buckets with water levels represented by a min-heap. In each operation, you choose the smallest non-zero water level, denoted by x, and remove x units of water from all non-empty buckets. Your task is to determine the minimum number of operations required to empty all the buckets. **You need to write the extract_min(), sink(), and the minimum_operation_find() methods /functions.**

Sample Input:	**Sample Output:**
heap = [1, 5, 9, 10, 12]	5

Explanation			
Heap Array	Heap Visual	Heap Array	Heap Visual
Initial Stage: [1, 5, 9, 10, 12]	**1** / \ **5 9** / \ **10 12**	**Step 3:** **Extracted the minimum value 4** [5, 7, 0, 0, 0] **Subtracted the minimum value (4) from each non-zero value** [1, 3, 0, 0, 0]	**1** / \ **3 0** / \ **0 0**
Step 1: **Extracted the minimum value 1** [5, 9, 10, 12, 0] **Subtracted the minimum value (1) from each non-zero value** [4, 8, 9, 11, 0]	**4** / \ **8 9** / \ **11 0**	**Step 4:** **Extracted the minimum value 1** [3, 0, 0, 0, 0] **Subtracted the minimum value (1) from each non-zero value** [2, 0, 0, 0, 0]	**2** / \ **0 0** / \ **0 0**
Step 2: **Extracted the minimum value 4** [8, 9, 11, 0, 0] **Subtracted the minimum value (4) from each non-zero value** [4, 5, 7, 0, 0]	**4** / \ **5 7** / \ / **0 0 0**	**Step 5:** **Extracted the minimum value 2** [0, 0, 0, 0, 0]	**0** / \ **0 0** / \ **0 0**

```
//Find and substract
int find(Integer[] heap) {
    int operations = 0;
    MinHeap h = new MinHeap(heap.length);
```

```
    for (int i = 0; i < tasks.length; i++) {
        h.insert(tasks[i]);
    }


    while (h.heapIsAllZero(h)==false) {
        int x = extract_min();
        if (x == 0) continue;

        for (int i = 0; i < size; i++) {
            if (heap[i] > 0)
                heap[i] = heap[i] - x;
        }

        swim();
        operations++;
    }
    return operations;
}



//Helper
boolean heapIsAllZero(MinHeap h) {
    for (int i = 0; i < size; i++)
        if (h[i] != 0) return false;
    return true;
}
```